Target Case Study

TARGET CASE STUDY

1. Company Overview
Target Corporation is a leading American retail corporation, headquartered in Minneapolis, Minnesota. It operates more than 1,900 stores across the United States, offering a wide range of products including apparel, home goods, electronics, groceries, and more. Known for its emphasis on affordable style, convenience, and customer experience, Target has positioned itself as a key player in the retail industry.

2. Business Model
Target follows a mass merchandise retail business model, blending physical stores with an expanding digital presence. It generates revenue through product sales, partnerships with brands, and exclusive product lines. The company integrates omnichannel strategies like Order Pickup, Drive Up, and Same Day Delivery to enhance customer convenience.

3. Competitive Advantage
Target differentiates itself through:
- Exclusive brands and designer collaborations.
- Strong supply chain and vendor relationships.
- Focus on customer experience and loyalty programs (e.g., Target Circle).
- Integration of digital and physical retail channels.

SQL QUERIES FOR ANALYSIS

----

1. Data type of all columns in customers:
```
DESCRIBE customers;
```

2. Time range of orders:
```
SELECT MIN(order_purchase_timestamp) AS first_order_date,
       MAX(order_purchase_timestamp) AS last_order_date
FROM orders;
```

3. Count cities & states of customers:
```
SELECT customer_city, customer_state, COUNT(*) AS total_customers
FROM customers
GROUP BY customer_city, customer_state
ORDER BY total_customers DESC;
```

4. Growing trend in number of orders:
```
SELECT YEAR(order_purchase_timestamp) AS order_year, COUNT(order_id) AS total_orders
FROM orders
```

```
GROUP BY YEAR(order_purchase_timestamp)
ORDER BY order_year;
```

5. Monthly seasonality:

```
SELECT MONTH(order_purchase_timestamp) AS month_num, COUNT(order_id) AS total_orders
FROM orders
GROUP BY MONTH(order_purchase_timestamp)
ORDER BY month_num;
```

6. Time of day for Brazilian customers:

```
SELECT CASE
    WHEN HOUR(order_purchase_timestamp) BETWEEN 0 AND 6 THEN 'Dawn'
    WHEN HOUR(order_purchase_timestamp) BETWEEN 7 AND 12 THEN 'Morning'
    WHEN HOUR(order_purchase_timestamp) BETWEEN 13 AND 18 THEN 'Afternoon'
    ELSE 'Night'
  END AS time_of_day, COUNT(order_id) AS total_orders
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_state = 'SP'
GROUP BY time_of_day;
```

7. Month-on-month orders per state:

```
SELECT c.customer_state, DATE_FORMAT(order_purchase_timestamp, '%Y-%m') AS order_month, COUNT(o.order_id) AS total_orders
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state, order_month
ORDER BY c.customer_state, order_month;
```

8. Customers distribution across states:

```
SELECT customer_state, COUNT(customer_id) AS total_customers
FROM customers
GROUP BY customer_state
ORDER BY total_customers DESC;
```

9. % Increase in order cost from 2017 to 2018 (Jan-Aug):

```
SELECT ((SUM(CASE WHEN YEAR(o.order_purchase_timestamp) = 2018 THEN p.payment_value ELSE 0 END) -
      SUM(CASE WHEN YEAR(o.order_purchase_timestamp) = 2017 THEN p.payment_value ELSE 0 END)) /
    SUM(CASE WHEN YEAR(o.order_purchase_timestamp) = 2017 THEN p.payment_value ELSE 0 END) * 100) AS percent_increase
```

```
FROM orders o
JOIN payments p ON o.order_id = p.order_id
WHERE MONTH(o.order_purchase_timestamp) BETWEEN 1 AND 8;
```

10. Total & Average order price per state:

```
SELECT c.customer_state, SUM(oi.price) AS total_price, AVG(oi.price) AS avg_price
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state;
```

11. Total & Average freight value per state:

```
SELECT c.customer_state, SUM(oi.freight_value) AS total_freight, AVG(oi.freight_value) AS avg_freight
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state;
```

12. Delivery time & estimated difference:

```
SELECT o.order_id,
    DATEDIFF(o.order_delivered_customer_date, o.order_purchase_timestamp) AS time_to_deliver,
    DATEDIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date) AS diff_estimated_delivery
FROM orders o;
```

13. Top 5 states with highest & lowest average freight:

```
-- Highest
SELECT c.customer_state, AVG(oi.freight_value) AS avg_freight
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY avg_freight DESC
LIMIT 5;

-- Lowest
SELECT c.customer_state, AVG(oi.freight_value) AS avg_freight
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY avg_freight ASC
```

```
LIMIT 5;
```

14. Top 5 states with highest & lowest average delivery time:

```
-- Highest
SELECT c.customer_state, AVG(DATEDIFF(o.order_delivered_customer_date, o.order_purchase_timestamp)) AS avg_delivery_time
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY avg_delivery_time DESC
LIMIT 5;

-- Lowest
SELECT c.customer_state, AVG(DATEDIFF(o.order_delivered_customer_date, o.order_purchase_timestamp)) AS avg_delivery_time
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY avg_delivery_time ASC
LIMIT 5;
```

15. Top 5 states with fastest delivery vs estimated:

```
SELECT c.customer_state, AVG(DATEDIFF(o.order_estimated_delivery_date, o.order_delivered_customer_date)) AS avg_days_early
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY avg_days_early DESC
LIMIT 5;
```

16. Month-on-month orders by payment type:

```
SELECT DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS order_month, p.payment_type, COUNT(DISTINCT o.order_id) AS total_orders
FROM orders o
JOIN payments p ON o.order_id = p.order_id
GROUP BY order_month, p.payment_type
ORDER BY order_month;
```

17. Orders by payment installments:

```
SELECT p.payment_installments, COUNT(DISTINCT o.order_id) AS total_orders
FROM payments p
JOIN orders o ON p.order_id = o.order_id
```

```
GROUP BY p.payment_installments
ORDER BY p.payment_installments;
```